

Planner Review

总体判断：这版 P0~P5 的主框架是合理的，已经比早期方案收敛很多；但不是所有 P 都同等成熟。P0、P5、P1 最接近可开发状态；P2 可作为低风险增强；P3 有明显过度承诺风险；P4 应保持后期 fallback。

文档最正确的总原则是：P5 是唯一 hard safety gate；P1/P2/P3/P4 只改变偏好、排序或初值，不承担安全裁决；P0 只提供数据，不改状态机。这和当前 EGO planner 的原始结构匹配，因为原始 planner 本质仍是 occupancy map + rebound B-spline optimizer + runtime collision check，并没有使用 integrity risk。

P0: RiskGridMap / PlannerIntegrityField

评价

总体合理，建议保留。

P0 的核心设计是正确的：

代码块

```
1  独立 buffer
2  同 world/map frame
3  不嵌入 EGO GridMap
4  P1/P2/P3/P4 用 queryCost()
5  P5 用 queryPredictedPL()
6  double-buffer + generation_id
```

这些边界非常重要。EGO `GridMap` 只应该继续表达 occupancy / inflated occupancy；RiskGridMap 表达的是定位完整性风险，两者语义、分辨率、更新频率都不同。文档中把 P0 定位为 optimizer / candidate scorer / reference bias / optional A* 的 risk lookup table，而不是新的 A* 地图，这个方向是对的。

主要问题

1. `query_time_s` 现在有点“假时空化”

文档接口里有：

```
1 queryCost(p_w, query_time_s)
2 queryPredictedPL(p_w, query_time_s)
```

但第 10 章的数据结构看起来仍然是一个 active buffer，而不是真正的 multi-horizon buffer。也就是说，目前设计更像：

代码块

```
1 single generation risk field + query_time_s 仅用于 age / log
```

如果没有明确 `horizon_layers`，P5 里 `now_s + tau` 查询未来 PL 会被误解成真的时变预测场。

修改建议

把 P0 明确成二选一，不能含糊：

推荐一步到位版本

代码块

```
1 struct RiskGridMapParams {
2     std::vector<double> horizons_s; // e.g. {0.0, 0.5, 1.0, 1.5, 2.0}
3 };
```

内部 buffer：

代码块

```
1 active_buffer[horizon_id][voxel_id]
```

查询逻辑：

代码块

```
1 tau = query_time_s - active_buffer_stamp_s
2 1. 在 horizons_s 中找相邻时间层
3 2. 每个时间层内做 3D trilinear interpolation
4 3. 时间层之间做 linear interpolation
```

如果暂时不想实现多 horizon，也要在文档里明确写：

代码块

```
1 RiskGridMap is quasi-static.
2 query_time_s only affects stale/age checking.
3 It does not provide true time-varying PL prediction.
```

否则 P5 的 future PL 语义会不严谨。

1. 命名建议

`RiskGridMap` 可以继续用，但我建议在文档中统一概念名：

代码块

```
1 PlannerIntegrityField = 抽象职责
2 RiskGridMap = 当前缓存实现
```

这样后续不会把它误解成 EGO 的另一个 map。最终对外接口应是：

代码块

```
1 queryCost()
2 queryPredictedPL()
3 health()
4 acquireSnapshot()
```

3. P1 优化期间必须固定 generation

P0 有 double-buffer 和 generation_id 是对的，但 P1 的 L-BFGS 优化期间不能每次 cost evaluation 都读“当前 active buffer”。如果优化过程中 `RiskGridMap` swap 了 generation，同一个控制点变量 `x` 的 cost 会突然变化，L-BFGS 会很难收敛。

应补充：

代码块

```
1 auto risk_snapshot = risk_grid.acquireSnapshot();
2
3 bspline_optimizer->setRiskSnapshot(risk_snapshot);
```

一次 `BsplineOptimizeTrajRebound()` 内部所有 P1 查询都使用同一个 snapshot。

P0 最终修改结论

保留 P0，但补三点：

1. 明确 single-layer 还是 multi-horizon; 推荐直接支持 horizons_s。
2. 文档中统一 PlannerIntegrityField / RiskGridMap 语义。
3. P1/P2/P3/P4 查询应支持 acquireSnapshot(), 避免优化中 generation 改变。

P5: Runtime Integrity Gate

评价

现在这版已经基本合理，明显比之前的 forward/lateral/backtrack recovery 设计更好。

P5 现在只输出：

代码块

```
1 OK
2 REQUEST_REPLAN
3 REQUEST_EMERGENCY_STOP_CANDIDATE
```

并且不自己生成 recovery target，不新增 FSM 状态，这非常正确。它与现有 EGO FSM 对齐：

REQUEST_REPLAN 映射到 REPLAN_TRAJ，emergency candidate 先尝试 planFromCurrentTraj()，失败才进入 EMERGENCY_STOP。文档也明确 P5 不是新的恢复规划器，而是只判断当前轨迹是否还能继续执行。

主要问题

1. current_invalid -> REQUEST_REPLAN 可能导致无限重规划

如果 /iap/integrity stale 或 invalid 是持续性的，P5 会反复：

代码块

```
1 EXEC_TRAJ -> REPLAN_TRAJ -> EXEC_TRAJ -> REQUEST_REPLAN
```

但这不是轨迹问题，而是 integrity 数据源问题。一直 replan 不能解决。

修改建议

增加一个 very simple stale escalation，不新增状态，只在 P5 内部计数：

代码块

```
1 p5:
```

```
2    current_stale_to_replan_s: 0.5
3    current_stale_to_emergency_s: 2.0
```

逻辑：

代码块

```
1    current_integrity stale < 0.5s:
2        OK_WITH_WARNING 或 REQUEST_REPLAN debounce
3
4    current_integrity stale >= 0.5s:
5        REQUEST_REPLAN
6
7    current_integrity stale >= 2.0s:
8        REQUEST_EMERGENCY_STOP_CANDIDATE
```

这样 debug 会更清楚：是 trajectory unsafe，还是 integrity source dead。

2. Future unknown 也需要 escalation

当前规则是：

代码块

```
1    unknown_ratio >= max_unknown_ratio -> REQUEST_REPLAN
```

这对单 tick 合理，但如果 RiskGridMap 长期 unknown，还是会无限 replan。

建议增加：

代码块

```
1    p5:
2        future_unknown_to_emergency_s: 2.0
```

也就是连续 unknown 过久才升级 emergency candidate。

3. P5 final gate 需要更明确

文档写了 final gate，但还需要规定失败后的行为：

代码块

```
1    GEN_NEW_TRAJ 中 final gate fail:
2        本次 plan attempt = failure
```

```
3    不发布轨迹
4    继续 GEN_NEW_TRAJ retry
5
6    REPLAN_TRAJ 中 final gate fail:
7    本次 replan attempt = failure
8    保持 REPLAN_TRAJ
9    若超过 retry/time budget, 则 EMERGENCY_STOP
```

否则会出现规划器生成了轨迹但不发布，FSM 却以为成功的问题。

P5 最终修改结论

P5 保留当前简化版，但补三点：

代码块

1. stale / unknown 持续时间升级机制。
2. final gate fail 的 FSM 返回语义。
3. status 中区分 current_invalid、future_bad、future_unknown、AL_invalid。

不要恢复 forward/lateral/backtrack，那会重新把 P5 变成 planner。

P2：候选轨迹排序

评价

合理，低风险，适合保留。

P2 的定位很清楚：只在 EGO 原生 `use_distinctive_trajs` 分支里，对已经优化成功的 candidate 做二次排序，不生成新同伦类，不做 hard safety。这个边界正确。文档中也明确 P2 不使用 raw PL、不计算 `PL < AL`，只使用 `RiskGridMap::queryCost()`，最终 candidate 仍必须经过 P5 gate。这个设计是干净的。

主要问题

1. P1 和 P2 可能重复计算 integrity cost

如果 P1 已经开启，那么 `BsplineOptimizeTrajRebound()` 的 `final_cost` 可能已经包含：

代码块

```
1    lambda_integrity * J_integrity
```

P2 又用：

代码块

```
1 candidate_score = optimizer_score + lambda_candidate_integrity *  
  integrity_score
```

这就会 double count integrity。

修改建议

明确规定 P2 使用的 `final_cost` 必须是不含 P1 的 original optimizer cost，或者 P2 在 P1 开启时采用分量化 cost：

代码块

```
1 struct OptimizerCostBreakdown {  
2     double total_cost;  
3     double original_cost;    // smooth + collision + feasibility + swarm +  
    terminal  
4     double integrity_cost;    // P1 only  
5 };
```

P2 评分应使用：

代码块

```
1 candidate_score =  
2     normalize(original_cost)  
3     + lambda_candidate_integrity * integrity_score
```

不要用 `total_cost`。

2. `final_cost / best_final_cost_seen` 归一化不够稳

如果 `best_final_cost_seen` 很小，或者不同场景 final cost 尺度变化大，归一化会不稳定。

建议改成当前候选集 min-max normalization：

代码块

```
1 optimizer_score =  
2     (final_cost - min_final_cost) /  
3     (max_final_cost - min_final_cost + eps)
```

如果只有一个成功候选，P2 不改变选择，只记录 metric。

P2 最终修改结论

保留 P2，但修改：

代码块

1. P2 使用 `original_cost`，不使用包含 P1 的 `total_cost`。
2. `final_cost` 归一化改成 `candidate-set min-max`。
3. P2 开发初期先只记录 `metrics`，不改变 `winner`；验证后再启用 `ranking`。

P1：后端完整性软代价

评价

总体合理，是最有价值的 `planner-level` 注入点之一。

P1 不再只采控制点，而是沿 B-spline 轨迹采样 `RiskGridMap::queryCost()`，再用 B-spline basis 把空间梯度反传回控制点。这是正确的。文档明确 P1 不做 `PL_pred < AL` hard constraint，只做低权重 preference，也不直接调用 Predictor，这些边界都合理。

主要问题

1. 必须固定 RiskGrid snapshot

这和 P0 的问题一样。P1 目前固定了：

代码块

```
1 integrity_query_base_time_s
```

这是对的，但还不够。还必须固定：

代码块

```
1 risk field generation
```

否则一次 L-BFGS 中，目标函数会随着 RiskGridMap 刷新而变。

修改建议

P1 接口改为：

代码块

```
1 void BsplineOptimizer::setRiskSnapshot(  
2     std::shared_ptr<const RiskGridSnapshot> snapshot,  
3     double query_base_time_s);
```

然后：

代码块

```
1 snapshot->queryCost(p, query_base_time_s + t, &sample);
```

2. `unknown_policy = skip` 合理，但要避免 optimizer 偏向 unknown 区

P1 默认跳过 unknown 是可以的，因为 P5 会 hard gate unknown。但如果大量 unknown 区域没有 cost，优化器可能没有风险梯度，仍然走向 unknown 区，然后被 P5 拒绝，导致反复 replan。

建议 P1 默认仍可 skip，但要加一个可选参数：

代码块

```
1 p1:  
2     unknown_policy: skip          # default  
3     unknown_policy_debug: small_penalty
```

并在 debug 中记录：

代码块

```
1 miss_count / sample_count
```

如果 P5 因 unknown 反复拒绝，就打开 small penalty。

3. `lambda_integrity = 1e-5` 不宜写死为推荐值

EGO 的原始 cost 尺度可能很大，`1e-5` 也可能完全不起作用。更好的写法是：

代码块

```
1 lambda_integrity 初始设小，但必须通过 grad_norm ratio 调参：  
2 weighted_grad_integrity_norm / grad_original_norm  $\approx$  5%~20%
```

也就是让 P1 成为二级偏好，而不是主导碰撞与动力学。

P1 最终修改结论

保留 P1，但补：

代码块

1. 固定 RiskGrid snapshot/generation。
2. 调参目标用 gradient ratio，而不是固定 lambda。
3. unknown skip 保持默认，但记录 miss/stale；必要时启用 small penalty。

P3：高完整性 Global Reference Bias

评价

想法合理，但当前设计过度承诺，是 P0~P5 中风险最大的部分。

P3 的动机是对的：如果 global reference 本身走向高风险走廊，后端 P1 很难完全拉出来。所以从 reference 层做走廊偏置是有意义的。

但当前 P3-global 有两个硬问题。

主要问题

1. Rolling RiskGridMap 不一定支持 global reference

P0 的 RiskGridMap 是以 UAV 为中心的 rolling local grid。P3-global 却想在 start-goal 全局路径上做 station sampling / beam search。如果 goal 超出 risk grid 范围，大部分 station 会 unknown。

这时 P3-global 实际上不能做 global corridor selection，只能做局部 reference bias。

修改建议

把 P3 拆成语义而不是拆类：

代码块

- 1 P3-local-reference-bias:
 - 2 默认支持
 - 3 基于当前 rolling RiskGridMap
 - 4 只在 local horizon 内偏置 reference / local target
- 5
- 6 P3-global-reference-bias:
 - 7 仅当存在 global or corridor-scale risk coverage 时启用

文档中“P3-global 是主线”这句话建议删除或改成：

代码块

- 1 P3-global is enabled only when the risk source covers the start-goal corridor.
- 2 With rolling local RiskGridMap only, P3 should operate as local reference bias.

2. Station + lateral beam search 不保证绕过复杂障碍

P3-global 的候选是围绕 start-goal 直线生成的 lateral samples。这个方法适合“双走廊横向偏置”这种 demo，但不适合复杂环境。如果障碍物需要大范围绕行，它可能找不到路径。

这不是不能做，但不能把它描述成通用 global planner。

P3 最终修改结论

P3 应保留为实验增强，但修改定位：

代码块

- 1 1. P3-local 可以保留，作为 getLocalTarget() 后的小范围 bias。
- 2 2. P3-global 只有在 risk field 覆盖整条 start-goal corridor 时启用。
- 3 3. P3-global 不能声称替代 obstacle-aware global planner。
- 4 4. 默认关闭，放在 P0/P5/P1/P2 稳定之后开发。

P4: Risk-aware Local A*

评价

设计合理，但价值有限，必须保持 optional fallback。

P4 的边界写得比较好：它只在 EGO 原始 `AStar::AstarSearch()` 的 neighbor expansion 中加入 risk edge cost，并保留 occupancy hard rejection。它不替代 P3-global，不计算 `PL < AL`，RiskGridMap 不 ready 时退回 original A*，这些都正确。

主要问题

1. P4 只在 collision segment 出现时生效

EGO 的 A* 不是主前端。它只在 `initControlPoints()` 发现 collision segment 后，用于生成 rebound guide path。文档前面也明确指出 A* 不是完整全局路径搜索，只是局部碰撞段 guide。所以如果当前轨迹没有碰撞，但完整性风险很高，P4 完全不会工作。这个能力边界必须写清楚。

2. risk-aware A* 可能影响 rebound 几何稳定性

A* guide path 是用来计算 `base_point / direction` 的。如果 risk cost 权重大，A* 可能生成绕得很远的 guide，导致 rebound direction 不稳定。

修改建议

P4 参数要更保守：

代码块

```
1 p4:
2   enable_risk_aware_astar: false
3   lambda_p4_risk: 0.05    # 不建议一开始 0.2
4   max_extra_path_ratio: 1.3
```

增加一条 hard fallback：

代码块

```
1 若 risk-aware A* path length > 1.3 * original A* path length:
2   use original A* path
```

这样可以防止 P4 为了低 risk 破坏局部绕障几何。

P4 最终修改结论

保留 P4，但明确：

代码块

```
1 1. P4 只影响 collision segment guide，不是全局走廊选择器。
2 2. 默认关闭。
3 3. 增加 path length ratio fallback。
4 4. 只有 P3/P2/P1 已经稳定后再测试。
```

P0~P5 整体结构评价

合理的部分

整体分层是对的：

代码块

- 1 P0：数据缓存
- 2 P5：hard safety gate
- 3 P1：后端 soft preference
- 4 P2：多候选 ranking
- 5 P3：reference-level bias
- 6 P4：local A* fallback

文档中反复强调 P5 是唯一 hard safety gate，P1/P2/P3/P4 不替代 P5，这是最重要的正确性边界。

最大的问题

当前文档还有一点“章节重复 + 最新口径覆盖旧口径”的痕迹。比如第 8 章是摘要接口，第 10~15 章是最新详细设计。这样给 Codex 开发时容易出现“到底按第 8 章还是第 10~15 章”的歧义。

建议修改文档结构

把第 8 章改成纯索引，不再保留旧伪代码和旧接口：

代码块

- 1 ## 8. P0-P5 Overview
- 2 只保留职责表、启用顺序、互斥边界。
- 3 详细设计见：
- 4 P0 -> Chapter 10
- 5 P5 -> Chapter 11
- 6 P2 -> Chapter 12
- 7 P1 -> Chapter 13
- 8 P3 -> Chapter 14
- 9 P4 -> Chapter 15

然后第 10~15 章作为唯一可执行 spec。